The following scenario, for example, won't work, because the `(fly|travel)` expression will be considered a regular expression, and Cucumber will try to pass it to one of the parameters. As a result, there will be too many parameters, and the step definition will break:

```
@When("^I (fly|travel) from (.*) to (.*) on (.*)$")
public void I_fly_from(String departure, String destination, Date date){
    // TODO
}
```

The first regular expression will match either "fly" or "travel".

Now there aren't enough parameters for all the regular expressions.

To get around this, you need to tell Cucumber that the first regular expression shouldn't be passed as a parameter. In regular expression terms, this is known as a *non-capturing* expression, and you can write it as follows:

```
@When("^I (?:fly|travel) from (.*) to (.*) on (.*)$")
public void I_fly_from(String departure, String destination, Date date) {
    // TODO
}
```

Now the first regular expression will be matched but not captured.

This will allow both variations, but will only pass the parameters you want to the step definition method.

6.5.4 Passing tables to steps

In Cucumber, you can use the `DataTable` class to pass tabular data to a step definition, which provides a rich API to extract, manipulate, and compare values in the table. But Cucumber also provides powerful built-in converters that can convert an embedded table to a list of domain objects.

Suppose you're writing Cucumber step definitions for the following scenario:

```
Scenario: Transfer points between existing members
  Given the following accounts:
    | owner | points | statusPoints  |
    | Jill  | 100000 | 800           |
    | Joe   | 50000  | 50            |
  When Joe transfers 40000 points to Jill
  Then the accounts should be the following:
    | owner | points | statusPoints  |
    | Jill  | 140000 | 800           |
    | John  | 10000  | 50            |
```

Now suppose you have an `Account` class that represents these fields:

```
public class Account {
    private final String owner;
    private final int points;
    private final int statusPoints;

    public Account(String owner, int points, int statusPoints) {
        this.owner = owner;
        this.points = points;
        this.statusPoints = statusPoints;
    }
```

You have a field for each table column.

This class represents rows in the scenario tables.

Cucumber knows how to instantiate an Account object for each row of the table using the constructor parameters.